

Simplified FaB Paxos in TLA+

Modeling and verifying Fast Byzantine Consensus

Team-7: Ion Tulei & Alexandru Verhovetchi

CS4720 Research in Program Analysis, TU Delft, 2026/Q4 · Project-A, Variant A

Paper: Martin and Alvisi, Fast Byzantine Consensus, IEEE TDSC 3(3), 2006

github.com/iontulei/CS4720-research-software-analysis-project-A

[Paper](#) › [algorithm](#) › [our model](#) › [results](#) › [related work](#) › [limits](#)

The problem, and the paper

Byzantine consensus

Agree on one value even when some processes lie.
Earlier Byzantine consensus needs three steps; PBFT, as a replicated state machine, takes four per request.

Proposers
propose

Acceptors
choose

Learners
learn

FaB Paxos (2006)

Two communication steps in the common case, with **$5f+1$** acceptors; the paper proves this bound tight.

- | | |
|------------|--|
| CS1 | only a proposed value is chosen |
| CS2 | only one value is chosen |
| CS3 | a correct learner learns only a chosen value |
| CL1 | some value is eventually chosen |
| CL2 | a chosen value is eventually learned |

Why TLA+, and our Research Questions

Model checking explores every interleaving; the dangerous ones are rare.

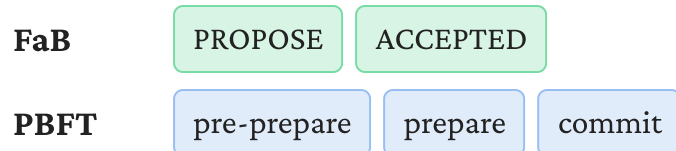
- 1 Can we build a faithful model under the full per-class Byzantine threat (one faulty acceptor, proposer, and learner)?
- 2 Do CS1-CS3 and CL1-CL2 hold under bounded model checking, and do the checks catch real errors - not just always pass?
- 3 How does the model-checking cost scale, and what keeps the model tractable?

Algorithm: the common case

Two steps: propose, accept, then a learner decides on a fast quorum.



The leader sends one message; each acceptor replies directly to all learners. Once a learner collects enough matching replies it decides - no further round-trip to the leader is needed. This is what makes FaB two steps instead of three.



Two thresholds matter ($f=1$, six acceptors). A learner needs **five** matching ACCEPTED messages to decide - the fast-learn quorum. A value is *chosen* once **four correct** acceptors have accepted it - the chosen threshold. The gap of one covers the single Byzantine vote that may be hiding inside the five.

Recovery and the progress certificate



Progress certificate: the new leader collects five signed replies from acceptors. The certificate confirms a value is safe to use if no competing value appears 3 or more times in those replies. If one value clearly dominates, the new leader must re-propose it; otherwise it may choose freely.

Poisonous write



When proposers time out, they suspect the leader and vote to replace it. The new leader then collects the certificate - and acceptors refuse to switch to any new value unless the certificate confirms it is safe.

A worked common-case run

The shortest run TLC found, eight states, from traces/common-case.

1. Leader **p1** proposes **v0** at proposal number 0 (PROPOSE)
2. Acceptors **a1 to a5** each accept and send ACCEPTED(**v0**, 0)
3. At the fifth matching ACCEPTED, the fast-learn quorum, learner **l1** decides (**v0**, 0)

One PROPOSE, five ACCEPTED, one decision. TLC generated this run automatically from the real unmodified protocol - we never wrote it by hand.

Our model: architecture



One protocol module, three thin wrappers, no duplicated logic.

- 7 state variables; one append-only message set `msgs`
- 5 message types: PROPOSE, ACCEPTED, LEARNED, QUERY, REP
- Each message carries its sender id, the signature model
- 9 actions: LeaderPropose, Accept, Learn, LearnViaPull, SuspectLeader, LeaderChange, Query, Reply, ByzantinePropose

```
Accept(a) ==
  /\ a \in CorrectAcceptors
  /\ \E m \in msgs :
    /\ m.type = "PROPOSE"
    /\ m.pn >= accLargestPN[a]
    /\ \/\ accAccepted[a] = None
       \/ VouchedAtPN(m.value, m.pn)
  /\ Send([type |->"ACCEPTED",
           acc |-> a, value |-> m.value,
           pn |-> m.pn])
```

Accept action in FaBPaxos.tla - the certificate check is the VouchedAtPN guard

Key design decisions

Certificate as a guard

PROPOSE carries only (value, pn); the guard reads signed REPs from msgs. Avoids a state blow-up.

Monotonic Chosen

Count ACCEPTED from correct acceptors in the append-only log, not a snapshot. Needed for CS3.

Byzantine baseline at Init

Inject every message a faulty acceptor or learner could ever send at startup. This covers the worst possible attacker and removes branching during the run.

Abstractions

One decision at a time (not a sequence of requests), abstract leader election, signatures as id stamps, no SATISFIED message.

Correctness as TLA+

```
Chosen(v, pn) ==  
  NumAcceptedMessagesCorrect(v, pn)  
    >= ChosenThreshold  
  
CS2Inv ==  
  \A v1, v2 \in Value, pn1, pn2 \in PN :  
    Chosen(v1, pn1) /\ Chosen(v2, pn2)  
      => v1 = v2
```

Why the quorums work



Fast-learn quorum = 5 ACCEPTED. Chosen = 4 correct (the blue minus a faulty vote). a1 is the one Byzantine acceptor (pink).

Why four and not three? Five votes minus one possible Byzantine vote = four guaranteed correct. If we only required three correct, two different quorums of five could overlap in just one honest acceptor - not enough to force both learners to the same value.

CS1-CS3 are invariants checked in every state; CL1-CL2 are temporal properties. CS3 quantifies over correct learners only.

A real bug we found and fixed

TLC caught a flaw in our own model, not in the protocol.

- Under recovery, TLC reported a CS3 violation on a 14-step trace
- Cause: our first `Chosen` read a snapshot of current state, not the durable log
- Fix: count `ACCEPTED` in the append-only `msgs`, so `Chosen` never reverts

Chosen definition	Result at MaxPN = 1
snapshot of <code>accAccepted</code> (buggy)	CS3 violated, 14-step trace
count over <code>msgs</code> (fixed)	clean, 147,340 states

Evaluation setup

Item	Value
Tools	Java 11+, TLC (tla2tools 2.19)
Acceptors	6, one Byzantine ($5f+1$)
Proposers / Learners	4 / 4, one Byzantine each
Values / Leader changes	2 / $\text{MaxPN} = 1$
Reductions	symmetry, buffer constraint

Deviations from the paper

Single-decree (one value, not a log), abstract leader election, signatures as identity stamps, no SATISFIED message. Sound for safety, stated as limitations.

Smallest config that still exercises a full Byzantine threat and one recovery. f is per class, and the paper lets one process play several roles; we model the classes as separate sets, one faulty agent each.

Verification results

Run	States	Result
Safety, MaxPN=1	147,340	clean
Liveness, CL1, CL2	4,642,112	clean
Safety, MaxPN=2	5,604,047	did not finish

CS1-CS3 and TypeOK hold across every reachable state at $f=1$. Liveness holds under fairness and a correct stable leader.

```
Safety run - MCFaBPaxos.cfg, 4 workers
Model checking completed.
No error has been found.
2,311,386 states generated
147,340 distinct states found
0 states left on queue.
Depth: 31    Finished in 10min 32s
```

Configuration	States
One value (instead of two)	18,316
Byzantine acceptor only	32,288
Baseline (two values, full Byzantine)	147,340
Symmetry off - did not finish	5,423,956+

Do the checks bite?

Mutation test

Reduce the votes needed to decide from 5 to 4 (weaken the fast-learn quorum). TLC finds a CS3 violation in under a second. This confirms the invariants catch real errors and do not just always pass.

Load-bearing test

Remove the certificate check, run with a Byzantine leader. TLC finds a CS2 violation in 14 seconds - the poisonous write attack succeeds. This confirms the certificate is the exact mechanism stopping that attack.

```
Load-bearing test - certificate check removed  
Error: Invariant CS2Inv is violated.
```

```
The following behaviour constitutes  
a counter-example:
```

```
State 1: <Init>
```

```
...
```

```
State 6: <ByzantinePropose>
```

```
←!— two different values chosen —→
```

```
Finished in 14s
```

CS2 violated in 14 s - poisonous write succeeds without the certificate

SysMoBench evaluation

Metric	Result
Syntax (SANY parses)	Pass
Runtime (TLC, 9 actions fire)	Pass
Conformance (vs implementation)	N/A
Invariant (CS, CL, non-vacuity)	Pass

We apply the metrics as a rubric. Conformance needs a running system; Variant A has none.

```
coverage, final block (spec/eval)
Accept ..... 13,214
Learn ..... 41,975
SuspectLeader ..... 31,452
Reply ..... 20,528
LeaderChange ..... 51
Query ..... 51
all 9 actions fire, no dead transitions
```

Related work

Protocol	Faults	Steps	Nodes $f=1$
Paxos	crash	2	3
PBFT	Byzantine	3	4
FaB (ours)	Byzantine	2	6
FaB tight (2021)	Byzantine	2	4

Steps are agreement steps; PBFT's full request is four.

Modeling: same certificate-as-guard idiom as Lamport's Paxos.tla.

Testing / traces: ByzzFuzz and SEFM trace validation are the dynamic counterparts to our static check.

Honest weak point: Kuznetsov et al. tightened the bound to $5f-1$, so FaB's 6 nodes are not minimal.

Limitations and threats to validity

f = 1 only. $f = 2$ needs eleven acceptors, out of reach for explicit-state TLC.

MaxPN = 2 unfinished. The clean exhaustive result is at one leader change.

Simplified subset. Abstract election, identity-stamp signatures, no SATISFIED message.

Superseded bound. We model the 2006 $5f+1$, not the tight $5f-1$.

No implementation, so no conformance and no code-level guarantee. The result is a proof for a bounded instance of a simplified protocol.

Questions?



Team-7 · Ion Tulei & Alexandru Verhovetchi · github.com/iontulei/CS4720-research-software-analysis-project-A